

Step 0: The empty skeleton

Branch `step-00-skeleton`

Goal Show where we start: a fresh Sulu 3.1 skeleton.

DO

```
bin/stand 0
```

SHOW

- The admin has no "Products" entry yet.

IF IT GOES WRONG

Typos: `git checkout -f step-00-skeleton` (instant, keeps the data). Stuck: `bin/stand 0 force` (the finished step with its data, about 30 seconds).

Step 1: Install and configure the bundle

Branch `step-01-install`

Goal Install the product bundle with three small changes and one config file.

DO

```
composer require sulu/product-bundle:3.0.x-dev
```

What it takes. On stage the Live block, no download.

`config/bundles.php`

```
Sulu\Product\Infrastructure\Symfony\HttpKernel\SuluProductBundle::class => ['all' => true],
```

`config/routes/sulu_admin.yaml`

```
sulu_product_api:
  resource: "@SuluProductBundle/config/routing_admin_api.yaml"
  prefix: /admin/api
```

`config/packages/sulu_product.yaml`

```
sulu_product:
  default_main_webpace: website
  association_types:
    accessory: { label: 'Goes well with' }
    alternative: { label: 'Alternatives' }
  measurements:
    weight: { units: [GRAM, KILOGRAM] }
    temperature: { units: [CELSIUS] }
    volume: ~ # every volume unit
```

The admin build is done before the talk and committed in `public/build/admin` (`npm install && npm run build` in `assets/admin`). Never on stage: a local build does not match the installed Sulu.

```
bin/adminconsole sulu:build dev
```

New `pr_*` tables and product rights for the admin role. On stage `bin/data 1` does it.

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-00-skeleton && git checkout step-01-install -- .
bin/data 1
```

SHOW

- Admin menu: Products, Attributes, Attribute groups, Product families.
- `sulu:build dev` AFTER the install: the admin role only gets product rights when it is created.

IF IT GOES WRONG

Typos: `git checkout -f step-01-install` (instant, keeps the data). Stuck: `bin/stand 1 force` (the finished step with its data, about 30 seconds).

Step 2: The catalogue as data

Branch `step-02-catalogue-data`

Goal Explain the entities with real data: group, attributes, families, products.

DO

`src/Catalogue/CatalogueData.php` (prepared, not typed)

```
'colour' => [
    'type' => 'options', 'name' => 'Colour', 'group' => 'Look', 'filterable' => true,
    'options' => ['teal' => 'Teal', 'mustard' => 'Mustard', 'black' => 'Black'],
],
'weight' => ['type' => 'number', 'name' => 'Weight', 'group' => 'Specs', 'config' => ['unit' =>
'GRAM']],
// families
'clothing' => ['name' => 'Clothing', 'attributes' => [
    'material' => ['required' => true],
    'colour' => ['variant' => true],
    'size' => ['variant' => true],
]],
// products
['code' => 'TS-1001', 'family' => 'clothing', 'title' => 'Hello World T-Shirt',
 'image' => 'hello-world-t-shirt-teal.jpg', 'attributes' => ['material' => 'organic-cotton',
 'weight' => 180], ...],
```

`vendor/bin/phpunit`

The test checks 27 products: every family, attribute, association and picture exists.

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-01-install && git checkout step-02-catalogue-data -- .
bin/data 2
```

SHOW

- Five attribute types: text, number with a unit, date, options, boolean (yes or no).
- Booleans here: `limited_edition` (filterable) and `dishwasher_safe`.
- 27 products, 41 photos in `data/images/`. Nothing is in the database yet.

IF IT GOES WRONG

Typos: `git checkout -f step-02-catalogue-data` (instant, keeps the data). Stuck: `bin/stand 2 force` (the finished step with its data, about 30 seconds).

Step 3: Import with a console command

Branch `step-03-import-command`

Goal Load the data with the same messages the admin uses.

DO

`src/Command/ImportCatalogueCommand.php`

```
$created = $this->dispatch(new CreateProductMessage([
    'locale'      => 'en',
    'template'    => 'product',
    'type'        => ProductInterface::TYPE_PRODUCT,
    'productFamily' => $familyIds[$product['family']],
    'code'        => $product['code'],
    'title'       => $product['title'],
    'attributes'  => $this->values($product['attributes'], $attributeIds), // attribute uuid
=> value
    'claim'       => $product['claim'],
    'url'         => $this->url($product['title']),
    'details'     => ['image' => ['id' => $this->image($product['image'],
$product['title'])]],
]));

$this->dispatch(new ApplyWorkflowTransitionProductMessage(['uuid' => $uuid], 'en', 'publish'));
```

`bin/adminconsole app:import-catalogue`

About a minute.

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-02-catalogue-data && git checkout step-03-import-command -- .
bin/adminconsole app:import-catalogue
bin/data 3
```

Run the import while the database is still empty, talk for a few seconds, then Ctrl+C: `bin/data 3` loads the complete data. Run again on full data, the import only says "The catalogue is already imported."

SHOW

- No preview and no product page yet: the Twig template comes in step 4 ("Page does not exist in html format"). Use it as the bridge to step 4.
- Admin: Products list, open one: Details (attributes), Content, SEO.
- Start the import first and talk while it runs.

IF IT GOES WRONG

Typos: `git checkout -f step-03-import-command` (instant, keeps the data). Stuck: `bin/stand 3 force` (the finished step with its data, about 30 seconds).

Step 4: The product page

Branch `step-04-product-page`

Goal Render a product on the website with Tailwind and Stimulus.

DO

```
composer require symfony/asset-mapper symfony/stimulus-bundle symfony/ux-turbo
symfonycasts/tailwind-bundle
bin/websiteconsole importmap:install
bin/websiteconsole tailwind:build
```

`templates/products/product.html.twig`

```
{% set groups = product.attributes|sulu_product_attribute_groups %}

<h1>{{ content.headline }}</h1>
<p>{{ content.claim }}</p>

{% for group in groups %}
    <h2>{{ group.label }}</h2>
    {% for attribute in group.attributes %}
        {{ attribute.label }}: {{ attribute.formattedValue }}
    {% endfor %}
{% endfor %}
```

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-03-import-command && git checkout step-04-product-page -- .
bin/data 4
```

SHOW

- Every picture has a small "AI" badge in the top right corner (`templates/products/_image.html.twig`): it reads the media's `origin`, the disclosure text is the tooltip.
- `/products/hello-world-t-shirt`: `unit` ("180 g"), date and option labels come formatted.
- A boolean comes as a real `true` or `false` in `formattedValue`: the template prints Yes or No. Example: a mug says "Dishwasher safe: Yes".

IF IT GOES WRONG

Typos: `git checkout -f step-04-product-page` (instant, keeps the data). Stuck: `bin/stand 4 force` (the finished step with its data, about 30 seconds).

Step 5: Associations

Branch `step-05-associations`

Goal Show accessories and alternatives on the product page, and show how little it takes to change what is resolved.

DO

`config/forms/product_associations.xml`

```
<property name="associations/accessory" type="product_selection">
  <meta><title lang="en">Goes well with</title></meta>
  <params>
    <param name="properties" type="collection">
      <param name="headline" value="headline"/>
      <param name="claim" value="claim"/>
      <param name="image" value="image"/>
    </param>
  </params>
</property>
```

`templates/products/product.html.twig`

```
{% for item in product.associations.accessory %}
  {% include 'products/_card.html.twig' %}
{% endfor %}
```

`templates/products/_card.html.twig`

```
<h3>{{ item.headline }}</h3>
<p>{{ item.claim }}</p>

```

Live change: delete the `claim` line, reload: the claim is gone from the cards. Add it again. Now add `<param name="code" value="code"/>` and print `{{ item.code }}` in the card.

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-04-product-page && git checkout step-05-associations -- .
bin/data 5
```

SHOW

- `/products/hello-world-t-shirt`: "Goes well with" and "Alternatives" under the product.
- Admin: the Associations tab of the same product.

IF IT GOES WRONG

Typos: `git checkout -f step-05-associations` (instant, keeps the data). Stuck: `bin/stand 5 force` (the finished step with its data, about 30 seconds).

Step 6: Colour and size variants

Branch `step-06-variants`

Goal One product, many variants: each with its own URL, colour, size and photo.

DO

`src/Catalogue/CatalogueData.php` (prepared)

```
'variants' => [
    ['code' => 'TS-1001-TEA-M', 'title' => 'Hello World T-Shirt, Teal, M',
      'image' => 'hello-world-t-shirt-teal.jpg', 'attributes' => ['colour' => 'teal', 'size' =>
        'm']],
  ],
```

`config/packages/sulu_product.yaml`

```
sulu_product:
  variants:
    properties:
      attributes: product.attributes
```

`templates/products/product.html.twig`

```
{% set variant = product.currentVariant|default(null) %}
{% set image = variant.image|default(product.image) %}

{% for option in product.variants %}
  <a href="{{ su_content_path(option.url) }}">
    {{ option.attributes.colour|sulu_product_format_attribute_value }}
  </a>
{% endfor %}
```

`config/forms/product_associations.xml` (a product with variants has no URL, so cards link to its first variant)

```
<param name="variants" value="product.variants"/>
```

`templates/products/_card.html.twig`

```
{% set url = item.url|default((item.variants|default([])|first).url|default(null)) %}
```

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-05-associations && git checkout step-06-variants -- .
bin/data 6
```

SHOW

- `/products/hello-world-t-shirt-teal-m` works, `/products/hello-world-t-shirt` does not.
- Click a colour: the photo changes. Click a size: the URL changes.

IF IT GOES WRONG

Typos: `git checkout -f step-06-variants` (instant, keeps the data). Stuck: `bin/stand 6 force` (the finished step with its data, about 30 seconds).

Step 7: Overview page with filters and search

Branch `step-07-catalogue-search`

Goal A catalogue page with filters and a search box. The URL is the whole state.

DO

`config/packages/sulu_product.yaml`

```
sulu_product:
  search:
    website:
      additional_product_filters: true
```

`src/Controller/Website/CatalogueController.php`

```
$search = $this->engine->createSearchBuilder('website')
    ->addFilter(Condition::equal('resourceKey', 'product'))
    ->addFilter(Condition::equal('locale', $locale));

$search->addFilter(Condition::search($term));
$search->addFilter(Condition::in('product.attributes_text_values', ['colour:teal']));
```

`assets/website/controllers/catalogue_filter_controller.js`

```
export default class extends Controller {
  static targets = ['form'];
  submit() { this.formTarget.requestSubmit(); }
}
```

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-06-variants && git checkout step-07-catalogue-search -- .
bin/data 7
```

SHOW

- `/products`: tick Teal, then Hoodie sizes M. The list changes.
- Tick "Limited edition: Yes": 6 products. A filterable boolean is one "Yes" checkbox, stored as `limited_edition:true` in the index.
- Type a typo, "hodie": Loupe still finds the hoodies. Elasticsearch does not out of the box.
- After a NEW filterable attribute: `cache:clear` and `cmsig:seal:reindex --index website --drop`.

IF IT GOES WRONG

Typos: `git checkout -f step-07-catalogue-search` (instant, keeps the data). Stuck: `bin/stand 7 force` (the finished step with its data, about 30 seconds). Never `bin/reset-demo` on stage: about 2 minutes.

Step 8: An AI client manages products (MCP)

Branch `step-08-mcp`

Goal Let an AI create a draft product, with the rights of a Sulu user.

DO

```
composer require sulu/mcp-bundle:1.0.x-dev
```

```
config/packages/sulu_mcp.yaml
```

```
sulu_mcp:
    dangerous_tools:
        product_write: true      # create and update drafts; publish stays off
```

```
NODE_EXTRA_CA_CERTS=~/.symfony5/certs/default.crt \
  claude mcp add --transport http sulu https://127.0.0.1:48400/admin/mcp
```

Prompt

Create the sticker "Rubber Duck Sticker" (code ST-9001) in the family Accessories, made of vinyl, a limited edition, with the URL `/products/rubber-duck-sticker` and the main webspace website, as a draft.

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-07-catalogue-search && git checkout step-08-mcp -- .
bin/data 8
```

SHOW

- Login as `ai-editor` / `ai-editor`, then the consent screen.
- Name the URL and the webspace in the prompt: the tool sets neither by itself.
- `ai-editor` may view the webspace `website` (role permission `sulu.webspaces.website`), otherwise the admin preview fails.
- Before the talk: `config/routes.yaml` imports only `App\Controller`, and `bin/reset-demo` has made the JWT keys (`config/jwt/`, not in git).

IF IT GOES WRONG

Show a recording of the same prompt. Typos: `git checkout -f step-08-mcp` (instant, keeps the data).

Stuck: `bin/stand 8 force` (about 30 seconds).

Step 9: Request for publishing

Branch `step-09-review-workflow`

Goal A product needs a review before it goes live. This is only configuration.

DO

`config/packages/sulu_content.yaml`

```
sulu_content:
  request_workflows:
    default:
      resources: [products]
      required_user_approvals: 1
      pre_validators:
        seo_required: { fields: [title, description] }
```

`config/routes/sulu_admin.yaml`

```
sulu_content_api:
  resource: "@SuluContentBundle/config/routing_admin_api.yaml"
  prefix: /admin/api
```

The product bundle already ties the review permission to `sulu.product.products`, so no security config is needed.

Flow: ai-editor "Save and request for publish" (SEO empty: refused) → fill SEO → request → locked → admin "Review", "Approve" → ai-editor "Publish".

LIVE

The code of this step on top of the previous step, so the IDE shows exactly its changes; then its packages, assets and data:

```
git checkout -q -f step-08-mcp && git checkout step-09-review-workflow -- .
bin/data 9
```

SHOW

- The button is on the Content, SEO and Excerpt tabs, not on Details.
- As admin the Review button shows after a page reload; open the product's Content tab fresh.

IF IT GOES WRONG

Typos: `git checkout -f step-09-review-workflow` (instant, keeps the data). Stuck: `bin/stand 9 force` (the finished step with its data, about 30 seconds).